

Upload files:

SRR29831631_1.fastq.gz

SRR29831631_2.[fastq.gz](#)

Converted

Uncompressed files

Quality Control

FastQC v0.12.1 (Galaxy)

Input:

Parameters default

Results:

high overall quality

Bias at first 10bp per base sequence content

Sequence length variation

Overrepresented sequences (Ns + minor technical sequences)

Adapter content low

Fastp v1.2.0 (Galaxy)

Input:

Paired uncompressed file

Mode: Paired-end

Parameters:

Adapter auto-detection (paired-end)

Quality filtering:

Qualified quality phred = 20

Unequalified percent limit = 40 (default)

N base limit = 5 (default)

Length filtering:

Minimum length = 50

Trimming:

Cut by quality in tail (3')

Window size = 4

Mean quality = 20

Overrepresented sequence analysis = ON

Remainder settings: Default

FastQC v0.12.1

Fastp paired outputs

SPAdes v4.2.0+galaxy0

Input: fastp paired output collection

Parameters:

Operation mode:

Assembly and error correction

Read type:

Paired-end (list of dataset pairs)

Orientation:

FR

Careful mode:

ON

Other parameters default

Filter FASTA

on the headers and/or the sequences

Galaxy Version 2.3

Criteria for filtering on the sequences

Sequence Length

Minimum length = 500

QUAST Genome assembly Quality

(Galaxy Version 5.3.0+galaxy1)

Assembly mode?

Individual assembly (1 contig file per sample)

Type of organism

Prokaryote

Prokka

Galaxy Version 1.14.6+galaxy1

Input: filtered scaffolds

Genus: Clostridium

Species: botulinum

Strain: SRR29831631

Kingdom: Bacteria

Genetic code: 11

Min contig: 200

Use genus DB: YES

Locus tag: SRR29831 (optional but recommended)

Abricate v1.0.1

Input: filtered scaffolds

Default parameters

NCBI BLAST+ blastn v2.16.0+galaxy0

was used in Galaxy to search bont reference sequences against the filtered scaffold assembly. Both megablast and blastn searches were performed. Megablast produced 0 hits, while blastn returned only short local alignments and no full-length bont gene hit.

CheckM2 1.1.0+galaxy0

was run on the filtered scaffolds to estimate completeness and contamination. Parameters on default.

The primary output saved was dataset 159, the CheckM2 quality report.

Galaxy tool ID: toolshed.g2.bx.psu.edu/repos/iuc/checkm2/checkm2/1.1.0+galaxy0

Barnap v1.2.2

was run on the filtered scaffolds to identify rRNA genes, including 16S rRNA.

Used the Bacteria setting and default parameters.

The saved outputs were dataset 162, the rRNA GFF file, and dataset 163, the rRNA sequence FASTA file.

toolshed.g2.bx.psu.edu/repos/iuc/barnap/barnap/1.2.2

FastANI v1.3

Comparing our scaffold to reference genomes of botulinum, sporogenes, and tagluense spp.

Query: filtered scaffold

Reference: clostridium_refs

Run with default parameters

Save tsv file as: ani_results.tsv

Now create a group with both your clostridium_refs and filtered scaffold

Name it: collection_allinall

Query: collection_allinall

Reference: collection_allinall

BASIC ANI HEATMAP WORKFLOW (fastANI → Python heatmap)

Navigate to project directory

```
cd /mnt/c/Users/prisc/OneDrive/Desktop/GalaxyAnalyses
```

Create output folder

```
mkdir -p ani_heatmap_galaxy
```

Confirm ANI results file exists

```
ls -l ani_results.tsv
```

4. Inspect file contents

```
head -n 5 ani_results.tsv
```

Check number of columns:

```
awk -F '\t' 'NR==1 {print "Columns:", NF}' ani_results.tsv
```

Expected: 5 columns

5. fastANI output format used

Columns:

1. query genome
2. reference genome
3. ANI
4. fragments matched
5. total fragments
6. Create Python script

```
nano ani_heatmap_galaxy/make_ani_heatmap.py
```

(Paste plotting script here)

7. Save and exit nano
8. Run script

```
python3 ani_heatmap_galaxy/make_ani_heatmap.py
```

9. Confirm output

```
ls ani_heatmap_galaxy
```

Expected:

```
ani_heatmap.png  
make_ani_heatmap.py
```

10. Open output (WSL)

```
explorer.exe ani_heatmap_galaxy
```

CLUSTERED ANI HEATMAP WORKFLOW

PART 1. Galaxy (all-vs-all ANI)

Upload all genome FASTA files (including your scaffold)

Create a dataset collection

Select all FASTA files

Click "Create Dataset Collection"
Choose "list"
Open FastANI (Galaxy Version 1.3)
Set parameters
Query Sequence(s) = collection
Reference Sequence(s) = same collection
Run FastANI
Check output
Column 1 should contain multiple different genomes

Download the output TSV
Save as:

ani_all_vs_all.tsv

PART 2. Local setup

Open terminal
cd /mnt/c/Users/prisc/OneDrive/Desktop/GalaxyAnalyses
Make output folder
mkdir -p ani_heatmap_galaxy
Confirm file exists
ls -l ani_all_vs_all.tsv
Preview file
head -n 10 ani_all_vs_all.tsv

PART 3. Create clustered script

Create new script
nano ani_heatmap_galaxy/ani_clustered_heatmap.py
Paste clustering script
Save and exit

PART 4. Install seaborn (if needed)

Check seaborn
python3 -c "import seaborn"
If needed
pip install seaborn

PART 5. Run clustering

Run script
python3 ani_heatmap_galaxy/ani_clustered_heatmap.py

Confirm output
ls ani_heatmap_galaxy

PART 6. Open result

Open folder
explorer.exe ani_heatmap_galaxy
Open:

Ani_clustered_heatmap.png

Versions used in terminal:

Python 3.13.9
pandas 2.3.3
matplotlib 3.10.7
Seaborn 0.13.2
Numpy 2.3.4
Scipy 1.16.2

PCA Plot Generation on MacBook

Directory used:

/Users/tyler/Desktop/GalaxyAnalyses

Step 1: Navigated to the Galaxy analysis folder:

```
cd ~/Desktop/GalaxyAnalyses
```

Step 2: Confirmed the project files were present:

```
ls
```

Step 3: Copied the final filtered scaffold assembly into the genome folder and renamed it as the query genome:

```
cp filtered_scaffold.fasta genomes/SRR29831631.fasta
```

Step 4: Checked the all-vs-all ANI file to confirm it contained fastANI output:

```
head ani_all_vs_all.tsv
```

Step 5: Checked that the ANI table included the filtered scaffold/SRR29831631 genome:

```
grep -i "SRR29831631\\|filtered\\|scaffold" ani_all_vs_all.tsv | head
```

Step 6: Created a Python PCA script:

```
nano make_pca_simple.py
```

Step 7: Ran the PCA script:

```
python3 make_pca_simple.py
```

Step 8: Opened the PCA output figure:

```
open ani_pca_simple.png
```

Step 9: Checked Python and package versions used for the PCA script:

```
python3 --version
```

```
python3 -c "import pandas, numpy, matplotlib, sklearn; print('pandas', pandas.__version__);  
print('numpy', numpy.__version__); print('matplotlib', matplotlib.__version__); print('scikit-learn',  
sklearn.__version__)"
```

Input file used for PCA:

```
ani_all_vs_all.tsv
```

Main output file generated:

```
ani_pca_simple.png
```

Versions used in Mac terminal:

```
Python 3.9.6
```

```
pandas 2.3.3
```

```
numpy 2.0.2
```

```
matplotlib 3.9.4
```

```
scikit-learn 1.6.1
```

CARD

RGI was run on `filtered_scaffold.fasta` using CARD RGI with criteria set to Perfect, Strict, and complete genes only. Results reported 0 perfect hits, 13 strict hits, and 0 loose hits.

Note: the final assembly was later renamed locally as `SRR29831631_filtered_scaffolds.fasta` for organization.

Final Galaxy dataset notes were saved in `methods_notes/galaxy_dataset_key.txt` to track the main dataset numbers used for preprocessing, assembly, annotation, toxin screening, ANI analysis, and downstream PCA visualization.